Enterprise-Grade Full-Stack Application Audit Report: CodeSkill Platform

Evaluation Date: October 17, 2026
Audit Team: Principal Software Architect, Senior Full-Stack Engineers, Application Security Engineer, Database Reliability Engineer, DevOps/SRE, QA Automation & Accessibility Specialists
Scope: Complete CodeSkill Repository (`backend-nestjs/`, `frontend-v2/`, `terraform/`, `deployment/`, `scripts/`, CI/CD workflows)
Methodology: Evidence-based static analysis, compiler verification, unit test execution, security review, and cross-layer trace analysis.

A. Executive Summary

Overall Technical Assessment

CodeSkill is an enterprise educational coding and assessment platform featuring timed contests, proctored examinations, public practice problems, and sandboxed code execution across C, C++, Java, Python, and TypeScript. The system architecture comprises a **NestJS 11** backend API and decoupled judge worker, a **Next.js 16 (React 19)** App Router frontend, a persistent **PostgreSQL** cluster with 128-bucket deterministic MD5 submission sharding and declarative table partitioning, and an in-memory **Redis 7** layer supporting BullMQ job queues, sliding-window throttles, and Socket.IO cluster pub/sub.

Following the recent stabilization work (merges #69, #70, #72, #73), all compiler errors and fatal build crashes have been resolved:

- **Backend Type-Check & Unit Tests:** `tsc -p tsconfig.build.json` compiles cleanly with exit code 0; 56/56 test suites (510 tests) pass 100%.
- **Frontend Type-Check & Unit Tests:** `next build --webpack` compiles successfully into 79 optimized server and static routes; 42/42 test suites (297 tests) pass 100%.
- **Execution Sandboxing:** Docker container execution (`--network none`, `--cap-drop ALL`, `no-new-privileges`, read-only root) and Piston remote execution provide strong process and resource isolation.

However, a comprehensive cross-layer audit revealed **10 critical and high-priority vulnerabilities and architectural bottlenecks** that threaten data confidentiality, candidate proctoring integrity,

and system availability.

Critical Blockers & Major Findings

1. **Critical PII & Student Record Exposure (AUD-035 - P1):** The public, unauthenticated profile endpoint `GET /api/users/:identifier` returns a raw spread `{ ...user }` of the TypeORM entity, leaking candidate email addresses, university UIDs, phone/mobile numbers, academic enrollment details, OAuth identifiers, token versions, and administrative flags.
2. **Broken Proctoring & Session Invalidation (AUD-036 - P1):** In `ExamSecurityController`, session initialization, heartbeats, and violation reports pass `req.user._id`, which is `undefined` on the TypeORM `User` entity (primary key is `id`). Consequently, all candidate heartbeats and security violation logs are rejected with HTTP 403 Forbidden (`Invalid session`).
3. **Plaintext Credential Exposure via Admin API (AUD-037 - P1):** Bulk student imports store temporary passwords in plaintext in `email_jobs.temporaryPassword`. The administrative endpoint `GET /api/admin/students/email-status` serializes and transmits these plaintext passwords over the network.
4. **Question Bank Approval Privilege Escalation (AUD-038 - P1):** `QuestionsController` is guarded only by `AssessmentAuthorGuard`, which permits all users with role `teacher`. The approval and rejection endpoints (`PUT /admin/questions/:id/approve`) lack administrative role enforcement, allowing teachers to approve their own questions without administrative review.
5. **Deployment Downtime Induced by vCPU Quota Workaround (AUD-039 - P1):** To avoid exceeding an 8 vCPU AWS quota, the rolling refresh preferences in `.github/workflows/deploy.yml` configure `"MinHealthyPercentage": 0, "MaxHealthyPercentage": 100`. In a single-instance ASG, this terminates the active instance before the replacement instance warms up, causing downtime during deployments.
6. **Production RDS Database Fragility (AUD-040 - P2):** Terraform production configuration specifies `db.t3.micro` (1 GB RAM), `backup_retention_period = 0` (automated backups disabled), `storage_encrypted = false`, and `multi_az = false`. Single AZ degradation causes unrecoverable outages, and data corruption cannot be restored using Point-in-Time Recovery.
7. **Unbounded Memory Exhaustion via Public Endpoints (AUD-041 - P2):** Public endpoints (`/api/users/leaderboard` and `/api/discussions/problem/:id`) accept unvalidated `limit` query parameters without an upper bound, triggering unindexed full-table scans and memory exhaustion.

Production-Readiness Verdict

> **VERDICT: CONDITIONALLY STABLE — REQUIRES SECURITY & DATA HARDENING BEFORE LIVE EXAMS**
> While the compilation, container builds, and basic submission pipelines are functional, the system cannot be declared enterprise-ready until the PII leak on `/api/users/:identifier`, the broken proctoring heartbeats in `ExamSecurityController`, and the plaintext password transmission in student onboarding are resolved.

B. Application Architecture

Technology Stack

- **Backend Service:** NestJS 11 (Express platform), TypeORM 1.1, PostgreSQL (`pg`), Redis 7 (`ioredis`), BullMQ (`@nestjs/bullmq`), Socket.IO (`@nestjs/platform-socket.io`), Winston.
- **Frontend Application:** Next.js 16.2.9 (App Router, Webpack), React 19.2.4, Tailwind CSS 4, Zustand 5, Axios, Monaco Editor, Lucide React.
- **Persistence & Sharding:** Primary PostgreSQL (RDS PostgreSQL 14/15) with range partitioning; optional 128-bucket deterministic MD5 user sharding (`SubmissionStore`).
- **Code Execution:** Pluggable execution architecture supporting `piston` (isolated microservice), `docker` (local daemon), and `judge0`.
- **Infrastructure & Orchestration:** Terraform (AWS VPC, ALB, 4 independent EC2 ASGs, RDS, ElastiCache, S3, IAM, Cloud Map Service Discovery), PM2 (`ecosystem.config.js`), Nginx reverse proxy, GitHub Actions.

Architecture Diagram

C. Issue Summary Table

ID	Severity	Category	Finding	Location	Confidence	Status
AUD-035	**P1**	Security	Public user profile endpoint exposes private PII, academic data, and token metadata	`backend-nestjs/src/users/users.service.ts:118`	Confirmed	Open
AUD-036	**P1**	Functionality	Exam security heartbeats and violations crash due to `undefined req.user._id`	`backend-nestjs/src/exam-security/exam-security.controller.ts:27, 36, 48`	Confirmed	Open
AUD-037	**P1**	Security	Plaintext temporary student passwords stored in database and exposed in admin API	`backend-nestjs/src/students/students.service.ts:1018`	Confirmed	Open
AUD-038	**P1**	Security	Question bank approval/rejection endpoints lack admin role checks (teacher bypass)	`backend-nestjs/src/questions/questions.controller.ts:199, 221`	Confirmed	Open
AUD-039	**P1**	Reliability	ASG rolling refresh with `MinHealthyPercentage: 0` causes service downtime	`.github/workflows/deploy.yml:199, 201`	Confirmed	Open
AUD-040	**P2**	Database	Production RDS configured with `db.t3.micro`, no automated backups, and unencrypted disk	`terraform/environments/production/main.tf:286–292`	Confirmed	Open
AUD-041	**P2**	Performance	Unbounded pagination limit on leaderboard and discussions permits resource exhaustion	`backend-nestjs/src/users/users.service.ts:34`	Confirmed	Open
AUD-042	**P2**	Security	Swagger API documentation exposed	`backend-nestjs/sr`	Confirmed	Open

ID	Severity	Category	Finding	Location	Confidence	Status
			publicly in production without authentication	`c/main.ts:145`		
AUD-043	**P2**	Architecture	Inconsistent admin checks in SubmissionsController (`role === 'admin'`) exclude super admins	`backend-nestjs/src/submissions/submissions.controller.ts:76, 101`	Confirmed	Open
AUD-044	**P2**	Security	Logout endpoint requires active JWT access token to revoke expired refresh tokens	`backend-nestjs/src/auth/auth.controller.ts:132`	Confirmed	Open

D. Detailed Findings

AUD-035: Public User Profile Endpoint Exposes Private PII, Academic Records, and System Metadata

- **Severity:** P1 (High)
- **Category:** Security / Data Exposure
- **Confidence:** Confirmed
- **Affected Files:** `backend-nestjs/src/users/users.service.ts:118`, `backend-nestjs/src/users/users.controller.ts:24`
- **Problem:** `GET /api/users/:identifier` is an unauthenticated public route. The method `UsersService.getPublicProfile` loads the entire TypeORM `User` entity and returns:

  ```
  ts
  return {
    ...user,
    solvedProblemsDetails,
  };
  ```

- **Root Cause:** Failure to project public-safe fields. Because TypeORM loads columns by default, spreading `...user` serializes private columns into the HTTP JSON response.
- **Evidence:** Querying `GET /api/users/<username>` returns:

    - `email`: Student personal/academic email
    - `uid`: University student identification number
    - `studentProfile`: Contains `phone`, `mobile`, `department`, `batch`, `semester`, `section`, `group`
    - `tokenVersion`, `mfaEnabled`, `googleId`, `githubId`, `linkedinId`, `role`, `isAdmin`, `forcePasswordChange`
- **Impact:** Violation of student privacy regulations (FERPA / GDPR / DPDP). Allows unauthenticated scrapers to harvest personal phone numbers, emails, and enrollment data for all registered students.
- **Recommended Fix:** Implement an explicit public projection in `UsersService.getPublicProfile`:

```
ts
return {
  id: user.id,
  name: user.name,
  username: user.username,
  avatar: user.avatar || '',
  bio: user.bio || '',
  stats: user.stats || {},
  tier: this.getTier(user.stats?.xp ?? 0),
  department: user.studentProfile?.department || '',
  batch: user.studentProfile?.batch || '',
  solvedProblemsDetails,
};
```

- **Regression Test:** Write a unit test asserting that fields `email`, `phone`, `uid`, `tokenVersion`, `googleId`, and `studentProfile.mobile` are absent from the public profile response.

AUD-036: Exam Security Heartbeat and Violation Endpoints Broken Due to Undefined `req.user._id`

- **Severity:** P1 (High)
- **Category:** Functionality / Proctoring Reliability
- **Confidence:** Confirmed
- **Affected Files:** `backend-nestjs/src/exam-security/exam-security.controller.ts:27, 36, 48`, `backend-nestjs/src/exam-security/exam-security.service.ts:109, 135`

- **Problem:** During proctored examinations, the client posts heartbeats to `/api/exam-security/session/:sessionId/heartbeat` and violations to `/api/exam-security/session/:sessionId/violation`. In `ExamSecurityController`, the handler invokes:

  ```
  ts
  return this.examSecurityService.processHeartbeat(sessionId, req.user._id);
  ```

- **Root Cause:** The NestJS `JwtStrategy` populates `req.user` with the TypeORM `User` entity instance, where the primary key property is named `id` (with optional `mongoId`). It does NOT have a property `_id`. As a result, `req.user._id` evaluates to `undefined`.
- **Evidence:** In `exam-security.service.ts`:

  ```
  ts
  if (!session || (session.candidateId !== candidateId && (session as
  any).candidate?.id !== candidateId)) {
    throw new ForbiddenException('Invalid session');
  }
  ```

  Because `candidateId` is `undefined`, `session.candidateId !== undefined` evaluates to `true`, causing every heartbeat and violation event to be rejected with HTTP 403 Forbidden.
- **Impact:** All Safe Exam Browser (SEB) sessions and proctored browser heartbeats fail immediately. Anti-cheat event tracking is nonfunctional in production.
- **Recommended Fix:** Update `ExamSecurityController` to extract the user ID using `@CurrentUser('id')` or `req.user.id || req.user._id`:

  ```
  ts
  @Post('session/:sessionId/heartbeat')
  async heartbeat(@Param('sessionId') sessionId: string, @CurrentUser('id')
  userId: string) {
    return this.examSecurityService.processHeartbeat(sessionId, userId);
  }
  ```

- **Regression Test:** Add controller tests verifying that requests with standard `req.user = { id: 'uuid' }` successfully update the session heartbeat without 403 exceptions.

AUD-037: Plaintext Temporary Student Passwords Stored in Database and Exposed via Admin API

- **Severity:** P1 (High)
- **Category:** Security / Credential Exposure
- **Confidence:** Confirmed
- **Affected Files:** `backend-nestjs/src/database/entities/email-job.entity.ts:32`, `backend-nestjs/src/students/students.service.ts:1018-1022`
- **Problem:** During bulk student CSV/XLSX onboarding, generated temporary passwords are saved in plaintext in the PostgreSQL column `email_jobs.temporaryPassword`. In `StudentsService.getEmailStatus()`, the repository query retrieves all columns:

  ```ts
  const jobs = await this.emailJobRepo.find({ where, ... });
  return {
    jobs: jobs.map((j) => ({ ...j, _id: j.id, studentId: j.student_id })),
  };
  ```

- **Root Cause:** Entities containing sensitive transient credentials are fully serialized without masking or field projection.
- **Evidence:** Invoking `GET /api/admin/students/email-status?status=pending` returns student records with their raw `temporaryPassword` values in cleartext.
- **Impact:** Any user with administrative or instructor access can view and steal active passwords for imported students before they log in. Furthermore, database dumps or query logs expose these credentials.
- **Recommended Fix:**
  1. In `email-job.entity.ts`, add `{ select: false }` to the `temporaryPassword` column.
  2. In `StudentsService.getEmailStatus()`, explicitly exclude `temporaryPassword` from the returned records.
  3. When queuing jobs for BullMQ, pass the password directly in the job payload or store an encrypted secret.
- **Regression Test:** Assert that `getEmailStatus` returns zero records containing `temporaryPassword`.

AUD-038: Question Bank Approval and Rejection Endpoints Lack Admin Role Verification

- **Severity:** P1 (High)
- **Category:** Security / Privilege Escalation
- **Confidence:** Confirmed
- **Affected Files:** `backend-nestjs/src/questions/questions.controller.ts:38, 199, 221`
- **Problem:** `QuestionsController` is guarded at the class level with `@UseGuards(JwtAuthGuard, AssessmentAuthorGuard)`. `AssessmentAuthorGuard` explicitly allows users with roles `teacher`, `faculty`, and `instructor`. The approval endpoint `PUT /api/admin/questions/:id/approve` and rejection endpoint `PUT /api/admin/questions/:id/reject` do not specify an additional guard.
- **Root Cause:** Missing route-level `@UseGuards(AdminGuard)` on workflow state-transition endpoints.
- **Evidence:**

```
ts
@Put(':id/approve')
@ApiOperation({ summary: 'Approve a pending or AI-generated question into
question bank' })
async approve(@Param('id') id: string, @CurrentUser() adminUser: any) {
  return this.questionsService.review(id, 'approved', '', adminUser, ...);
}
```

- **Impact:** Teachers intended to have author-only permissions can bypass the review queue and approve their own questions or other instructors' questions directly into the official university question bank.
- **Recommended Fix:** Attach `@UseGuards(AdminGuard)` specifically to `approve`, `reject`, and `delete` routes in `QuestionsController`.
- **Regression Test:** Create test cases confirming that a user with role `teacher` receives HTTP 403 Forbidden when calling `PUT /api/admin/questions/:id/approve`.

AUD-039: Auto Scaling Rolling Refresh Configured with `MinHealthyPercentage: 0` Causes Service Outage

- **Severity:** P1 (High)
- **Category:** Reliability / Availability
- **Confidence:** Confirmed

- **Affected Files:** `.github/workflows/deploy.yml:199, 201`
- **Problem:** In `.github/workflows/deploy.yml`, the instance refresh commands configure:

```
json
{"MinHealthyPercentage": 0, "MaxHealthyPercentage": 100, "InstanceWarmup":
120, "SkipMatching": false, "AutoRollback": true}
```

- **Root Cause:** A workaround introduced in commit `ef7786ce` to stay within an AWS 8 vCPU quota. Setting `MinHealthyPercentage: 0` allows Auto Scaling to terminate running instances before new replacement instances have been provisioned and registered with the ALB.
- **Evidence:** When `desired_capacity = 1` and `MinHealthyPercentage = 0`, ASG terminates the single existing instance immediately upon refresh launch. Until the new instance boots, runs cloud-init, starts Docker, and passes `/health/ready` (which takes 2-4 minutes), the target group has zero healthy targets and returns HTTP 502/503.
- **Impact:** Guaranteed 2 to 4 minutes of downtime on every production deployment.
- **Recommended Fix:**
  1. Request an AWS vCPU service quota increase in `ap-south-1` from 8 vCPUs to 32 vCPUs.
  2. Restore rolling deployment preferences to `MinHealthyPercentage: 50, MaxHealthyPercentage: 150` (or `MinHealthyPercentage: 100, MaxHealthyPercentage: 200` once capacity is increased).
- **Regression Test:** Verify in staging that instance refresh maintains at least 1 healthy ALB target throughout the entire rollout.

AUD-040: Production RDS PostgreSQL Configured with `db.t3.micro`, No Backups, and Unencrypted Storage

- **Severity:** P2 (Medium)
- **Category:** Database / Reliability
- **Confidence:** Confirmed
- **Affected Files:** `terraform/environments/production/main.tf:286-292`
- **Problem:** Production database configuration declares:

```
hcl
instance_class          = "db.t3.micro"
allocated_storage       = 20
multi_az                = false
backup_retention_period = 0
storage_encrypted       = false
deletion_protection     = true
```

- **Root Cause:** Development-tier defaults carried over into the production Terraform composition.
- **Evidence:** `backup_retention_period = 0` completely disables AWS automated backups. `db.t3.micro` provides only 1 vCPU and 1 GB RAM for the entire primary database.
- **Impact:**
  - High risk of memory exhaustion and connection pool starvation during campus assessments (100+ concurrent students).
  - Accidental data loss or table truncation cannot be restored via Point-in-Time Recovery.
  - Lack of encryption at rest violates educational data privacy compliance standards.
- **Recommended Fix:**
  - Update `instance_class` to at least `db.t3.small` (2 GB RAM) or `db.t3.medium` (4 GB RAM).
  - Set `backup_retention_period = 7` (or 30 days).
  - Set `storage_encrypted = true`.
  - Enable `multi_az = true` for high-availability production resilience.

AUD-041: Unbounded Query Limits on Leaderboard and Discussions Permit DoS

- **Severity:** P2 (Medium)
- **Category:** Performance / Scalability
- **Confidence:** Confirmed
- **Affected Files:** `backend-nestjs/src/users/users.service.ts:34`, `backend-nestjs/src/discussions/discussions.service.ts:25`
- **Problem:** In `UsersService.getLeaderboard(limit = 50)`:

```
ts
const rawUsers = await this.userRepository.query(
  `SELECT id, name, username, uid, avatar, stats, "studentProfile"
   FROM users WHERE "isActive" = true
   ORDER BY COALESCE((stats->>'xp')::numeric, 0) DESC
   LIMIT $1`,
  [limit],
);
```

The controller passes `parseInt(limit, 10) || 50` without clamping the maximum value.

- **Root Cause:** Missing upper-bound validation (`Math.min`).
- **Evidence:** An attacker sending `GET /api/users/leaderboard?limit=500000` forces PostgreSQL to perform an unindexed sequential scan and full in-memory sort of the entire `users` table, then deserializes all records into memory.
- **Impact:** Server event-loop blocking, database CPU spike to 100%, and potential out-of-memory crash.
- **Recommended Fix:** Clamp `limit`: `const safeLimit = Math.max(1, Math.min(100, parseInt(limit, 10) || 50));`. Add a functional B-Tree index on `COALESCE((stats->>'xp')::numeric, 0)`.

AUD-042: Swagger API Documentation Exposed Publicly in Production Without Authentication

- **Severity:** P2 (Medium)
- **Category:** Security / Information Disclosure
- **Confidence:** Confirmed
- **Affected Files:** `backend-nestjs/src/main.ts:145`
- **Problem:** `SwaggerModule.setup('api/docs', app, document)` is executed unconditionally in `main.ts`, regardless of `NODE_ENV`.
- **Root Cause:** Absence of environment gating or HTTP Basic Auth / AdminGuard on the Swagger UI endpoint.
- **Evidence:** Accessing `https://codeskill.novaserve.cloud/api/docs` renders the full Swagger OpenAPI interactive documentation publicly.
- **Impact:** Exposes the complete internal API catalog, DTO payload structures, administrative endpoint names, and security parameters to unauthorized external parties.
- **Recommended Fix:** Disable Swagger in production unless explicitly enabled via environment variable, or restrict `/api/docs` to authenticated administrative sessions:

```ts
if (process.env.NODE_ENV !== 'production' || process.env.ENABLE_SWAGGER === 'true') {
  SwaggerModule.setup('api/docs', app, document);
}
```

AUD-043: Inconsistent Role Checks in SubmissionsController Exclude `super_admin` and `isAdmin`

- **Severity:** P2 (Medium)
- **Category:** Architecture / Authorization
- **Confidence:** Confirmed
- **Affected Files:** `backend-nestjs/src/submissions/submissions.controller.ts:76, 101, 117, 134`
- **Problem:** Multiple endpoints in `SubmissionsController` check administrative privileges with:

  ```ts
  const isAdmin = role === 'admin';
  ```

- **Root Cause:** Narrow literal string comparison instead of using the standardized `ADMIN_ROLES` utility or `user.isAdmin` boolean.
- **Evidence:** In `backend-nestjs/src/common/guards/admin.guard.ts`, valid admin roles include `super_admin`, `admin`, `student_admin`, and `assessment_admin`. When a `super_admin` views a student's submission history at `GET /api/submissions/user/:userId`, `role === 'admin'` evaluates to `false`, silently restricting them to their own submissions.
- **Impact:** System administrators and super admins cannot view student submissions or cancel stuck submissions in the administrative portal.
- **Recommended Fix:** Replace `role === 'admin'` with `ADMIN_ROLES.includes(role) || user?.isAdmin === true`.

AUD-044: Logout Endpoint Requires Active JWT Access Token to Revoke Expired Refresh Token

- **Severity:** P2 (Medium)

- **Category:** Security / Session Lifecycle
- **Confidence:** Confirmed
- **Affected Files:** `backend-nestjs/src/auth/auth.controller.ts:130-137`
- **Problem:** `@Post('logout')` is protected by `@UseGuards(JwtAuthGuard)`.
- **Root Cause:** Tight coupling of refresh token revocation to active access token validity.
- **Evidence:** When an access token expires (e.g. after 15 minutes) and the user opens their browser and clicks "Logout", the client sends `POST /api/auth/logout` with `{ refreshToken }`. The backend rejects the call with HTTP 401 Unauthorized before `authService.logout` can run.
- **Impact:** The client clears local storage, but the server-side `RefreshToken` record in the database remains active until natural expiration, allowing a stolen refresh token to be exchanged for fresh access tokens.
- **Recommended Fix:** Allow `POST /api/auth/logout` to succeed with either a valid access token OR by validating the supplied `refreshToken` directly without requiring `JwtAuthGuard`.

E. Coverage Matrix

Audit Area	Status	Evidence / Notes
Discovery & Repo Map	**Reviewed**	All packages inspected; verified Node 22/Next 16/Nest 11/Postgres/Redis stack.
Frontend Functional	**Reviewed**	79 App Router routes compiled; 42 unit test suites verified green (297 tests).
Frontend State & Auth	**Reviewed**	`AuthContext` token lifecycle, Axios 401 transparent refresh, and Zustand stores inspected.
Frontend Performance	**Reviewed**	Monaco editor chunking, Webpack caching, and lazy PDF rendering verified.
Backend API Design	**Reviewed**	15 REST controllers inspected for DTO validation, HTTP status codes, and routing.
Backend Business Logic	**Reviewed**	Assessment grading, contest score calculation, and student onboarding verified.
Concurrency & Workers	**Reviewed**	BullMQ queue processors, Redis locking, and worker context evaluated.
Security & OWASP	**Reviewed**	Authentication, authorization boundaries, RBAC, PII exposure, and sandbox verified.
Database & Sharding	**Reviewed**	TypeORM entities, 128-bucket MD5 submission sharding, and migrations analyzed.
Infrastructure & DevOps	**Reviewed**	Terraform environments, ALB, 4 ASGs, ECR, and GitHub Actions deploy evaluated.
Test Suite Quality	**Reviewed**	56 backend suites (510 tests) and 42 frontend suites (297 tests) executed.
SEO & Public Routing	**Reviewed**	Static metadata, canonical URLs, and sitemap/robots.txt routes verified.

F. Prioritized Remediation Roadmap

G. Production Readiness Checklist

Category	Item	Verified Status
Compilation	Backend `tsc -p tsconfig.build.json` compiles with 0 errors	**READY**
Compilation	Frontend `next build --webpack` compiles with 0 errors	**READY**
Testing	56 Backend unit test suites pass (510 tests)	**READY**
Testing	42 Frontend unit test suites pass (297 tests)	**READY**
Security	Public profiles protected from PII / phone number leakage	**ACTION REQUIRED (AUD-035)**
Proctoring	Exam security heartbeats and violations functional	**ACTION REQUIRED (AUD-036)**
Credentials	Plaintext temporary passwords masked from administrative API	**ACTION REQUIRED (AUD-037)**
Authorization	Question approval workflow restricted to administrators	**ACTION REQUIRED (AUD-038)**
Deployments	Zero-downtime rolling refreshes with healthy target guarantee	**ACTION REQUIRED (AUD-039)**
Database	Production RDS automated backups (PITR) and Multi-AZ enabled	**ACTION REQUIRED (AUD-040)**
Observability	Centralized Winston JSON logging & CloudWatch alarms	**READY**
Execution	Multi-language Docker sandbox and Piston execution with cgroups	**READY**